

query. The way to determine the next vertex in the stack is the same as in the proof of Lemma C.27. We will omit the simple but tedious detailed construction here. $\square$

C.10 Proof of Theorem 5.4

In this section, we will prove Theorem 5.4. We will first restate the theorem for convenience.

Theorem 5.4. *Given $A, B > 0$, for all polynomial-time Turing machines $T \in \text{TIME}(n^A)$ with B states and vocabulary size B , there exist Linear RNNs with B special symbols, $O(A \log n)$ -bit precision and memory, and $O(AB^2)$ parameters that can output the result of T by running $O(n^A)$ steps of CoT with In-context RAG.*

Proof of Theorem 5.4. We will denote the state of T as $1, \dots, B$ (we will use 1 as the initial state) and the vocabulary of T as $1, \dots, B$. We will assume T operates on an infinite tape TAPE, which is a sequence of cells indexed by $\mathbb{Z}$. We will also assume that the tape is initialized with all cells being 0 except for the n cell starting at 1. The Turing machine also has a pointer p that points to a cell in the tape. The pointer is initialized to 1. At each time step, the Turing machine reads the cell pointed by POINTER and updates the cell pointed by POINTER and the pointer p according to the transition function $\delta : [B+1] \times [B] \rightarrow [B] \times [B] \times \{-1, 1\}$, which takes the current state and the current cell value (could be empty, which corresponds to $B+1$) as input and outputs the new state, the new cell value and the direction to move the pointer. The Turing machine halts when the state is B . Because $T \in \text{TIME}(n^A)$, the Turing machine will always halt in n^A steps. We will use $\text{TAPE}[t, i]$ as the value on the i -th cell on TAPE before the t timestep. We will use $\text{POINTER}[t]$ as the value of the pointer before the t timestep and $\text{State}[t]$ as the state of the Turing machine before the t timestep. We will further use $\text{Direction}[t]$ as the direction of the pointer movement before the t timestep.

We will first define the sequence that the retrieval-augmented RNN will generate and then construct an RNN that can generate such a sequence.

Sequence generation. The input token sequence $\mathcal{S}_{\text{in}}$ will be as followed,

$$\mathcal{S}_{\text{in}} = \langle s \rangle, \text{TAPE}[1, 1], \text{TAPE}[1, 2], \dots, \text{TAPE}[1, n]$$

Here all the symbols on the tape are represented by one special symbol in the vocabulary. Given this input token sequence, the retrieval augmented RNN will generate the following output token sequence,

$$\begin{aligned} \mathcal{S} = & \mathcal{S}_{\text{in}}, \langle \text{StartSearch} \rangle, ^{(\cdot)} (? : \backslash S \backslash s *) \cdot \{ 1 \} (\backslash S), \langle \text{EndSearch} \rangle, \\ & \text{TAPE}[1, 1] \\ & 1, \text{TAPE}[1, 1], 1, \\ & \dots \\ & \langle \text{StartSearch} \rangle, ^{(\cdot)} (? : \backslash S \backslash s *) \cdot \{ n \} (\backslash S), \langle \text{EndSearch} \rangle, \\ & \text{TAPE}[1, n] \\ & n, \text{TAPE}[1, n], n, \\ & \langle \text{StartSearch} \rangle, ((\text{POINTER}[1] \cdot (.) \text{POINTER}[1] \cdot * ? \$)), \langle \text{EndSearch} \rangle, \\ & \text{SearchResult}(1), \\ & \text{POINTER}[1], \text{TAPE}[2, \text{POINTER}[1]], \text{POINTER}[1] \\ & \text{State}[2], \text{Direction}[2], \\ & \dots \\ & \langle \text{StartSearch} \rangle, (\text{POINTER}[t] \cdot (.) \text{POINTER}[t] \cdot * ? \$), \langle \text{EndSearch} \rangle, \\ & \text{SearchResult}(t) \\ & \text{POINTER}[t], \text{TAPE}[t+1, \text{POINTER}[t]], \text{POINTER}[t], \\ & \text{State}[t+1], \text{Direction}[t+1], \end{aligned}$$

Here $\text{SearchResult}(t)$ is defined as

$$\text{SearchResult}(t) = \begin{cases} \text{FAILED}; & \text{if } \text{POINTER}[t] \text{ is empty cell before } t \\ \text{TAPE}[t, \text{POINTER}[t]]; & \text{otherwise} \end{cases}$$

The output token sequence simulates the Turing machine T on the tape TAPE due to the following lemma.

Lemma C.31. *Given any $t \in [n^A]$ and $i \in [n^A]$, the last string in $\mathcal{S}$ that contains i , i as a substring is $i, \text{TAPE}[t, i], i$ if $\text{TAPE}[t, i]$ is not empty and is the empty string otherwise.*

Proof. The proof is by induction, for $t = 1$, the result holds. For any $t \geq 2$, we only need to notice that $\text{POINTER}[t-1]$ is the only cell that can be updated at time $t - 1$. $\square$

Construction of RNN Given the input, the RNN can first iterate over 1 to n and generate the first n search queries and results by maintaining a counter in its state and memorizing the most recent search result (Lemma C.20). Then it is easy to see that the retrieval oracle will generate the correct $\text{SearchResult}(t)$ given the input $\mathcal{S}$. Therefore, we will only need to construct an RNN that can generate the rest part of $\mathcal{S}$.

We will assume the RNN maintains the state and pointer of the Turing machine in its state and show that they can be updated.

Based on Lemma C.20, the RNN can maintain constant recent token types in its state, we will assume the RNN memorize the last tokens up to the most recent $\langle \text{StartSearch} \rangle$ and also calculate the position relative to the most recent $\langle \text{StartSearch} \rangle$. By a lookup table in the FFN Lemma C.4, the RNN can output the fixed format of the search query. Similarly, RNN can output the $\text{POINTER}[t]$. To generate the update $\text{TAPE}[t + 1, \text{POINTER}[t], \text{State}[t], \text{Direction}[t]$, the RNN can use a FFN with $O(B^2)$ width to memorize the transition function of the Turing machine and output the update. Then, the RNN can use the memorized recent input to update the state and the pointer of the Turing machine at the next $\langle \text{StartSearch} \rangle$. The proof is then complete. $\square$

C.11 Proof of Theorem 5.6

Theorem 5.6. *For task $T \in \{\text{Index}, \text{AR}, c\text{-gram retrieval}, \text{Counting}, \text{IsTree}\}$, there exist constant-size hybrid Linear RNNs that can solve T . For T other than IsTree , no CoT is required and for IsTree , $O(n \log n)$ steps of CoT is required.*

Proof. The proof here is essentially the same as the construction of the Transformer in Theorem 4.6. We would use the same Transformer layer to solve T . The only difference is that we would use the output of the RNN, instead of FFN, as the input of the Transformer layer. Also for Counting, instead of using a COPY function, we write the query token in the state of the RNN (Lemma C.20). $\square$

C.12 Proof of Theorem 5.7

Theorem 5.7. *There exists a hybrid Linear RNN with $O(\log n)$ bit memory and $O(\log n)$ parameter, that can solve IsTree of size n with a chain of thought of length $O(n \log n)$.*

Proof. The proof is similar to the proof of Theorem 5.3. However, instead of using regular expressions to retrieve the next neighbor and parent, we will need to use the Transformer layer. The Transformer layer can retrieve the parent through an attention head implementing the match closest head (Lemma C.16) if the RNN part maintains the predecessor of each node in the chain of thought.

Retrieving the next neighbor is more complicated and we will use $O(\log n)$ steps of the chain of thought to do that. Given an edge (v, v') , we will first use one match head to retrieve the position p of (v, v') in the input sequence and write it to the chain of thought. Then we will use two MatchClose heads to retrieve the edge that contains v and is closest to $p + 2^i$ for $i = 0, 1, \dots, \log_2 n$ iteratively until the heads return an edge that is not (v, v') or i reaches $\log_2 n$. Here 2^i can be computed through doubling one of the dimensions in the state of the RNN and reset that dimension to 1 after termination. We will then compare the retrieved edge with the father of v to check if it is the same. If it is the same, we will search the next neighbor of v after the parent of v in the same way. The other part of the proof is similar to the proof of Theorem 5.3. $\square$

C.13 Proof of Theorem 5.8

Theorem 5.8. *Given any constant A, B , for any polynomial-time Turing machine $T \in \text{TIME}(n^A)$ with B states and vocabulary size B , there exists a hybrid Linear RNN (see Definition 5.5) with vocabulary of B special symbol, $O(A \log n)$ bit precision and memory, and $O(AB^2 \log n)$ bit parameters, that can simulate the result of T on any input with length n in $O(n^A)$ CoT steps.*

Proof. **Sequence Generation.** Under the same formulation of proof of the Theorem 5.4. The hybrid RNN will output the following sequence.

$$\begin{aligned} \mathcal{S} = & \mathcal{S}_{\text{in}}, \text{POINTER}[1], \text{TAPE}[2, \text{POINTER}[1]], \text{POINTER}[1] \\ & \text{State}[2], \text{Direction}[2], \\ & \dots \\ & \text{POINTER}[t], \text{TAPE}[t+1, \text{POINTER}[t]], \text{POINTER}[t], \\ & \text{State}[t+1], \text{Direction}[t+1], \end{aligned}$$

Note that Lemma C.31 still holds. We only need to prove that the hybrid architecture can generate the above sequence.

Hybrid Construction. The way RNN maintains the pointers and the states is the same as the proof of Theorem 5.4. Given each pointer value, we can retrieve the last value of the cell at the pointer through the one layer of attention by implementing a match closest head (Lemma C.16). $\square$